

Aihrly — Junior Backend Developer (NSS) Take-Home Assessment

Hi, and thanks for your interest in joining Aihrly.

This take-home is designed to give us a realistic picture of how you think and how you code — not to trick you or stress you out. It's scoped for **roughly 6–8 hours** of focused work. Don't spend a week on it. We'd rather see honest work you're proud of, with notes on what you'd do next if you had more time.

If anything is unclear, make a reasonable assumption, document it in your README, and keep moving. That's exactly what you'd do on the job.

The Scenario

Aihrly is an Applicant Tracking System (ATS). You're building the **team-side pipeline** — the piece of the product our customers (recruiters and hiring managers) actually live inside all day.

The flow is this: a company posts a job. Candidates apply. Then the **hiring team** works those applications through a pipeline — leaving notes, scoring candidates on different dimensions, and moving them from stage to stage. Every action is attributed to the team member who did it, because hiring is a team sport and accountability matters.

You're going to build a **tiny slice** of that system as a standalone ASP.NET Core Web API, backed by PostgreSQL.

This is deliberately small. We care more about *how* you build it than *how much* you build.

Part 1 — The API (required)

Entities you'll need

- **TeamMember** — (`id`), (`name`), (`email`), (`role`) (e.g. `recruiter`, `hiring_manager`). Seed 2–3 of these at startup so we have actors to work with. No auth — we'll identify the acting team member via an `X-Team-Member-Id` header on every mutating request.
- **Job** — (`id`), (`title`), (`description`), (`location`), (`status`) (`open` / `closed`).
- **Application** — belongs to a job, has a candidate (name, email), a current (`stage`), and the three scores below. A candidate cannot apply to the same job twice with the same email.
- **ApplicationNote** — belongs to an application, has a (`type`), (`description`), (`created_by`) (team member), and (`created_at`).

- **StageHistory** — records every stage change: `from_stage`, `to_stage`, `changed_by`, `changed_at`, optional reason/comment.

Pipeline stages and valid transitions

```

applied      -> screening, rejected
screening    -> interview, rejected
interview    -> offer, rejected
offer        -> hired, rejected
hired        -> (terminal)
rejected     -> (terminal)

```

An invalid transition (e.g. `applied → hired`) must return `400` with a clear error message.

Endpoints

Jobs

- `POST /api/jobs` — create a job.
- `GET /api/jobs` — list jobs. Filter by `?status=open`. Paginate with `?page=1&pageSize=20`.
- `GET /api/jobs/{id}` — get a single job.

Applications

- `POST /api/jobs/{jobId}/applications` — a candidate applies (name, email, optional cover letter). New applications start in `applied`. This is the one public-ish endpoint and does **not** require `X-Team-Member-Id`.
- `GET /api/jobs/{jobId}/applications` — list applications for a job. Filter by `?stage=screening`.
- `GET /api/applications/{id}` — **full applicant profile**: basic info, current stage, all three scores, all notes (with author name), and full stage history. This is the endpoint the pipeline UI calls most often — think about what the frontend would actually want in one round-trip.
- `PATCH /api/applications/{id}/stage` — move the application to a new stage. Body includes the target stage and an optional comment/reason. Requires `X-Team-Member-Id`. Must:

- validate the transition against the rules above,
- record a row in `StageHistory` with `from`, `to`, who, when, and the reason.

Notes

- `POST /api/applications/{id}/notes` — add a note. Body: `type` and `description`. `createdBy` comes from the `X-Team-Member-Id` header, **not** the body (do not trust clients to tell you who they are). Valid note types: `general`, `screening`, `interview`, `reference_check`, `red_flag`.
- `GET /api/applications/{id}/notes` — list notes for an application, newest first. Include the author's name, not just their id.

Scores — three separate endpoints, one per dimension

- `PUT /api/applications/{id}/scores/culture-fit` — body: `{ "score": 1-5, "comment": "..."}`
- `PUT /api/applications/{id}/scores/interview` — body: `{ "score": 1-5, "comment": "..."}`
- `PUT /api/applications/{id}/scores/assessment` — body: `{ "score": 1-5, "comment": "..."}`

All three require `X-Team-Member-Id`. Store the score, who set it, when, and the comment. `PUT` semantics — submitting a second time for the same dimension overwrites the previous value (but see Part 3 question 2b).

Must-haves

1. **ASP.NET Core** (.NET 8 or 9).
2. **PostgreSQL** via **EF Core** with **migrations**. We should be able to run your migrations on a fresh database and get a working schema. Npgsql is the expected driver.
3. **DTOs separate from entities**. Don't return EF entities directly from controllers.
4. **Validation** on incoming requests — required fields, valid email, valid enum values, score between 1 and 5. Return `400` with a useful message.
5. **Consistent error responses**. Pick a shape and stick to it. `application/problem+json` is a solid default.
6. **Reject missing/invalid `X-Team-Member-Id`** on endpoints that require it — return `400` or `401` (your call, be consistent), never silently accept it.
7. **At least 4 automated tests** covering something meaningful. For example:
 - A unit test for the stage-transition rules (valid + invalid).
 - An integration test that creates an application, adds a note, and reads it back — confirming the author name is resolved correctly.

- A test that submits a score twice and confirms the second value wins, and that `updated_by` / `updated_at` reflect the second submission.
- A test that proves the duplicate-application rule (same email + same job → rejected).

8. A **README.md** with:

- How to run it locally (with or without Docker).
- How to run the tests.
- How to seed team members, or which IDs exist by default.
- Any assumptions you made.
- Anything you'd improve with more time.

What we're looking for

- Clean, readable code. Small methods. Sensible folder structure.
 - Async/await used correctly. No `.Result` / `.Wait()` on async calls.
 - Thoughtful use of DI, no static state.
 - A schema that makes sense — correct types, indexes on what you'd actually query (e.g. `application_id` on notes and stage history), foreign keys, and `NOT NULL` where appropriate.
 - Commits that tell a story. 8 focused commits beats 1 giant "initial commit".
-

Part 2 — Pick ONE deep-dive (required — just one)

Choose the one you're most interested in. Do not attempt all of them.

Option A — Background job

When an application moves into `hired` or `rejected`, schedule a background job (Hangfire, `BackgroundService`, or a simple queue — your call) that simulates sending a notification email. It should write a log line and insert a row into a `notifications` table (`id`, `application_id`, `type`, `sent_at`). The `PATCH /stage` endpoint must return immediately — the notification must be dispatched asynchronously.

Option B — Caching with Redis

Cache `GET /api/applications/{id}` (the full profile) in Redis with a short TTL (e.g. 60 seconds). Invalidate the cache whenever a note, score, or stage change is written for that application. In your README, explain **when** this cache helps and when it could hurt a real recruiter's experience.

Option C — Multi-tenancy

Every Job, Application, TeamMember, Note, and StageHistory row belongs to an `organization_id`. Every request must carry an `X-Organization-Id` header. An org must only see its own data — including when resolving `X-Team-Member-Id` (a team member from Org B shouldn't be usable in Org A). Write a test that proves Org A cannot read or mutate Org B's applications.

Whichever you pick, write 3-5 sentences in your README explaining your design decisions and trade-offs.

Part 3 — Short written answers (required)

In a file called `ANSWERS.md`, answer these briefly. Short and honest beats long and waffly.

- Schema question.** Sketch (in SQL or plain text) the tables for `applications`, `application_notes`, `stage_history`, and however you chose to store the three scores. Call out your indexes and why. Specifically: what does the query for `GET /api/applications/{id}` look like against your schema, and how many round-trips does it take?
 - Scoring design trade-off.** You just built three separate endpoints, one per score dimension. Answer both:
 - (a) Why might three separate endpoints be better than one generic `PUT /api/applications/{id}/scores` that accepts all three dimensions in one body? And when would the opposite be true?
 - (b) If the product team said tomorrow *"we need to see the history of every score change, not just the current value"*, how would your schema change? Would the endpoints change?
 - Debugging question.** A recruiter reports: *"I moved a candidate to Interview yesterday and today the system says they're still in Screening."* You have access to logs, the database, and the client (browser). Walk us through, step by step, how you'd investigate. (5-10 bullet points.)
 - Honest self-assessment.** On a scale of 1-5, rate yourself on: **C#, SQL, Git, REST API design, Writing tests.** One sentence per rating. No wrong answers — we're calibrating, not grading.
-

Nice-to-haves (optional, do NOT stretch for these)

Only touch these if you finish the above comfortably.

- A working `Dockerfile` and `docker-compose.yml` that brings up the API + Postgres (+ Redis if you did Option B).
 - A simple GitHub Actions workflow that runs your tests on push.
 - Swagger / OpenAPI with meaningful summaries and example requests.
 - A seed script or endpoint that creates a job, a few applications, and a couple of notes so we can play with the API right away.
-

What we are NOT looking for

- A frontend. There is no UI required.
- Real authentication. Header-based actor identification (`X-Team-Member-Id`) is enough. Skip JWT, OAuth, Identity, etc.
- Production hardening. This is a take-home, not a deployment.
- Over-engineering. No Clean Architecture + CQRS + MediatR ceremony for this size of API. Keep it proportional.

If you find yourself building abstractions "in case we need them later" — stop. We value simplicity at this stage.

Submission

Push your solution to a **public GitHub repository** (or a private one with `@aihrly-hiring` invited — we'll send the handle when you start).

Reply to the hiring email with:

1. The repo link.
2. A 3–5 sentence summary of what you built and what you'd do next with more time.
3. The approximate number of hours you spent. Be honest — we use it to calibrate, not to judge.

If you get stuck and can't resolve something, **submit anyway** with a note in the README explaining what you tried and where you got blocked. We'd much rather see that than silence.

Evaluation rubric

Area	Weight
Correctness — does it actually work end-to-end?	30%
Code quality, readability, structure	20%
Database & schema design (esp. notes, scores, stage history)	15%
Tests — meaningful, not coverage theatre	15%
Written answers & communication (README, ANSWERS.md, commits)	15%
Your chosen deep-dive (Part 2)	5%

Nice-to-haves are tiebreakers, nothing more.

Good luck. We're genuinely rooting for you.

— The Aihrlly Tech Team